

VIDEO_GEN_PROVIDER

HelixLLM Video-Generation Provider — Design Spike (P3-T3 / GPU BURST tier — the heaviest burst)

Status	DESIGN (spike before implementation, §11.4.6 — do NOT code the provider until this is agreed AND a full GPU burst window is scheduled; see §0)
Scope	The GPU video-generation capability — the heaviest Burst-tier (single-owner §11.4.119) workload on the single RTX 5090 · 32 GB. Video diffusion models are large AND very compute-heavy — most need the FULL card (coder paused) and occupy it for minutes, not seconds
Owns	Implementation-plan item P3-T3 (04_implementation_plan.md:81 — “Video gen (WAN 2.2 + LTX): ComfyUI; async job endpoint. Signature: ffprobe frame-advance + codec/resolution/fps (§11.4.107) on the produced mp4.”)
Created	2026-07-07 · Revision 1 · Track (T1/main) · Branch feature/helixllm-full-extension
Grounding	submodules/helix_llm/docs/VRAM_BROKER.md §2 (Burst tier: “WAN/LTX video-gen”) + internal/vrambroker/broker.go+errors.go (the a12df57 broker CORE that now EXISTS: Acquire/Release/Budget, ClassVideo, single-owner + fail-closed over-budget refusal) · submodules/helix_llm/docs/API_CONTRACT.md §2/§4 (gateway /v1

group) · docs/research/
07.2026/00_master/
04_implementation_plan.md P3-T3 ·
sibling **IMAGE_GEN_PROVIDER.md**
(commit **299cf2c4** — same GPU-
burst pattern, same broker, same
ComfyUI engine, same async-job +
self-validated-analyzer discipline;
this doc is its video counterpart)

Anti-bluff (§11.4.6): every VRAM / latency / wall-clock-per-second-of-video / step-count figure in this document that is not a directly-quoted upstream value is an **estimate to be measured** ((EST – measure)). `nvidia-smi deltas + wall-clock` captured under `docs/qa/<run-id>/video_gen/` are the real budget; no benchmark below is a captured measurement. Every co-reside-vs-burst verdict **MUST** be re-confirmed against a live Budget () . free read before any PASS. **Video is heavier than image** — where FLUX-schnell is 2–8 s (image sibling §1.5), a video clip is **tens of seconds to over an hour** on consumer cards (§1.5 below), so the async-job route (§3) is not optional and the single-owner burst occupancy is long.

0. Why this is DESIGN-only, and why it is the HEAVIEST BURST workload (honest §11.4.6)

The embeddings (P3-T6) and translation (P3-T5) CPU spikes ship **before** the P0/P1 GPU chain because they take a **zero-byte VRAM lease** (VRAM_BROKER.md §2 CPU-only tier). Image generation (P3-T2) is a Burst member. Video generation is the **canonical, heaviest Burst-tier member** (VRAM_BROKER.md §2 names it explicitly: *“Burst (single-owner §11.4.119) ... FLUX image-gen, WAN/LTX video-gen ... started per-job, never co-resident with another burst; may require pausing warm tier”*).

The live-state constraint (the reason this is DESIGN, not impl): the RTX 5090 (32 GB) currently holds the resident coder fleet — **~19.4 GB used, ~12.7 GB free** (VRAM_BROKER.md §2 resident tier ≈ 18 GB weights + 10–12 GB KV). The flagship (full-quality) video models do NOT fit that free residual, and — unlike image models — they are also **very compute-heavy**, so a single clip occupies the card for **minutes to over an hour** on consumer cards:

- **WAN 2.2 T2V-A14B / I2V-A14B (the MoE quality lane)** upstream states it must *“run on a GPU with at least 80GB VRAM”* for the reference config (§1.3) — it does not fit a 32 GB card at all without fp8/GGUF quantisation, let alone the free 12.7 GB, and the reference I2V-A14B run is *~1 hr 20 min on an RTX 4090* (EST via community reports).

- **HunyuanVideo (13B 3D-DiT)** is “a 13B 3D DiT model requiring 47–58 GB VRAM at FP16” (§1.3) — the heaviest of the three, and non-commercially licensed.
- **LTX-Video 13B-dev** upstream’s own system-requirements floor is “NVIDIA GPU with a minimum 32GB+ VRAM — more is better” (§1.3).

So the flagship/quality path requires a **full GPU burst** — pausing the warm/coder tier to free the card — which is an **operator-gated** decision (VRAM_BROKER.md §2: “may require pausing warm tier”). We will **not** take that decision now; this document is the **exact, ready-to-implement spec** so that when a burst window is scheduled, implementation is a mechanical follow-through, not a fresh design.

A **smaller, heavily-quantised / distilled fast lane DOES co-reside** in the free ~12.7 GB (§1.5) — that lane makes video-gen available even when no burst window is scheduled, and is the recommended first thing to actually build. It is tighter than the image case (video’s text-encoder + denoiser + VAE each consume VRAM independently), so co-residency is offload/quantisation-dependent and **MUST** be confirmed against a live Budget () . free.

Honest boundary vs the plan (§11.4.6).

04_implementation_plan.md:81 specifies P3-T3 as “Video gen (WAN 2.2 + LTX): ComfyUI; async job endpoint. Signature: ffprobe frame-advance + codec/resolution/fps (§11.4.107) on the produced mp4.” This document (a) keeps ComfyUI + WAN 2.2 + LTX-Video as the reuse choice (§11.4.74), (b) **strengthens** the “ffprobe frame-advance” signature into a full §11.4.107 liveness battery (freeze-detection + independent frame-advance) PLUS a semantic prompt → video correspondence oracle (per-frame CLIPScore / VLM-caption) with a golden-bad self-validation (§5, per §11.4.107(10) — an ffprobe “non-zero frame count” check alone is a §11.4 bluff: a static/black or wrong-content clip passes it), and (c) adds the explicit Burst-tier broker integration (ClassVideo) + the async-job route the plan line names but does not spell out. Flagged, not a contradiction.

1. Engine choice — evidence-based comparison

1.1 The candidates

#	Engine + model	Serves video-gen how?	Fits free ~12.7 GB (co-reside)?	Coverage	New engine to the tree?
A	ComfyUI + WAN 2.2 (Wan-Video/Wan2.2)	Graph server:	T12V-5B: yes-TIGHT (~8 GB	T2V, I2V, TI2V;	Container only (no Go

#	Engine + model	Serves video-gen how?	Fits free ~12.7 GB (co-reside)?	Coverage	New engine to the tree?
		POST / prompt → /ws progress → / history/{id} → / view (SAME engine + routes as image-gen \$11.4.74)	w/ ComfyUI native offload); A14B MoE: no → full burst	480P/720P; MoE quality	dep); the plan names it; ComfyUI already the image-gen backend
B	ComfyUI + LTX-Video (Lightricks/LTX-Video)	Same ComfyUI graph API	2B-distilled fp8+tiling: yes (~6–8 GB); 13B-dev: no → burst	DiT T2V+I2V; up to 4K/50fps; distilled = fast	Same ComfyUI engine — different model weights only
C	ComfyUI + HunyuanVideo (tencent/HunyuanVideo)	Same ComfyUI graph API	fp8+temporal-tiling ~8 GB (reduced res) but SLOW; fp16 47–58 GB → burst	13B 3D-DiT T2V	Same ComfyUI engine; non-commercial licence (\$1.4)
D	diffusers (WanPipeline/LTXPipeline/HunyuanVideoPipeline + a FastAPI shim)	Python lib; needs a custom async-job shim	same footprints as A–C, per-quant	every HF video pipeline + every offload knob	New Python shim + heavier deps; ComfyUI already covers A–C
E	Hosted API (OpenAI Sora /v1/videos, Runway, others)	Native async-job — but a bespoke	N/A (remote)	remote	New remote dep + secret + egress ; OpenAI's Sora Videos

#	Engine + model	Serves video-gen how?	Fits free ~12.7 GB (co-reside)?	Coverage	New engine to the tree?
		remote adapter			API is itself deprecating (§3 note) — rejected as default

1.2 Decision

Primary engine: ComfyUI serving WAN 2.2 (the plan’s named model), on a full GPU burst for the quality lane (operator-gated) with a co-reside-tight lighter lane. ComfyUI is the SAME reuse backend the image-gen sibling already stands up (§11.4.74) — one engine, one set of routes (POST /prompt → /ws → /history/{id} → /view), one async gateway route (§3), swapping only the model weights. Two WAN 2.2 lanes behind that one backend:

- **WAN 2.2 TI2V-5B (Apache-2.0) — the default lighter lane.** A “5B dense model” with a “High-compression VAE, T2V+I2V, supports 720P” that ComfyUI documents as fitting “well on 8GB vram with the ComfyUI native offloading” — so it **co-resides TIGHT** in the free ~12.7 GB (offload-dependent, §1.5), giving a real video lane with no coder pause. Apache-2.0 licence (commercial-clean).
- **WAN 2.2 T2V-A14B / I2V-A14B (Apache-2.0 MoE) — the higher-quality lane.** The “27B total / 14B active” two-expert MoE (“high-noise expert ... low-noise expert”); reference config “at least 80GB VRAM” → **full GPU burst (coder paused, operator-gated)** with fp8_scaled quantisation to fit the 5090’s 32 GB.

Fast/lighter fallback: ComfyUI serving LTX-Video 2B-distilled (OpenRAIL-M) — the fast lane, which CO-RESIDES in the free ~12.7 GB and is the fastest of all three. LTX-Video is “the first DiT-based video generation model”; its 2B-distilled variant is “15× faster” and “real-time capable”, generates “720x480x121 videos in under a minute on RTX 4060 (8GB VRAM)” — so it fits the free residual with the broker’s 2 GiB headroom, needs **no** coder pause, and returns a clip in **tens of seconds** rather than minutes. **Recommendation: build the co-reside fast lane FIRST** (LTX-2B-distilled and/or WAN TI2V-5B — available today, no operator gate), and land the full-burst WAN A14B / LTX 13B quality lanes when a burst window is scheduled (§0 / §2.3).

HunyuanVideo is compared and rejected as the default (§1.3(6)) — heaviest footprint (47–58 GB fp16) AND non-commercially licensed. It remains a config-

selectable model behind the same ComfyUI backend for a research/quality burst, never the default.

1.3 Justification (cited, LATEST upstream docs — §11.4.99 / §11.4.150)

1. **WAN 2.2 is the right primary: the plan names it, it is Apache-2.0 (commercial-clean), and it spans a co-reside lane + an MoE quality lane behind ONE ComfyUI backend.** Upstream: TI2V-5B is a “5B dense model” with “High-compression VAE, T2V+I2V, supports 720P”; the MoE lanes are a “Text-to-Video MoE model, supports 480P & 720P” / “Image-to-Video MoE model, supports 480P & 720P” where “each expert model has about 14B parameters, resulting in a total of 27B parameters but only 14B active parameters per step” via a “two-expert design ... a high-noise expert for the early stages ... and a low-noise expert for the later stages”, all “Licensed under the Apache 2.0 License”. Source: [Wan-Video/Wan2.2 \(GitHub README\)](#) (accessed 2026-07-07).
2. **WAN 2.2’s VRAM/compute wall is why the quality lane is a full burst, quantified from upstream.** TI2V-5B “run[s] on a GPU with at least 24GB VRAM (e.g, RTX 4090 GPU)” and “can generate a 5-second 720P video in under 9 minutes on a single consumer-grade GPU”; the A14B MoE lanes “run on a GPU with at least 80GB VRAM”. The 24 GB / 80 GB reference floors both exceed the free 12.7 GB — the 5B fits only via ComfyUI’s native offloading, the A14B only via a full burst + fp8 quantisation on the 5090’s 32 GB. And “under 9 minutes” for one 5-second clip is the evidence for “video occupies the burst for minutes.” Source: [Wan-Video/Wan2.2 \(GitHub README\)](#) (accessed 2026-07-07).
3. **ComfyUI is a real programmatic backend for WAN 2.2, with a documented co-reside lane.** ComfyUI’s own WAN 2.2 tutorial specifies the required files — 5B: wan2.2_ti2v_5B_fp16.safetensors + wan2.2_vae.safetensors + umt5_xx1_fp8_e4m3fn_scaled.safetensors; 14B T2V: the two experts wan2.2_t2v_high_noise_14B_fp8_scaled.safetensors + wan2.2_t2v_low_noise_14B_fp8_scaled.safetensors — and states “The Wan2.2 5B version should fit well on 8GB vram with the ComfyUI native offloading.” The text encoder is umt5-xx1 (offloadable to CPU, \$1.5). ComfyUI exposes the same four routes the image sibling bridges (POST /prompt, /ws, GET /history/{id}, GET /view). Sources: [ComfyUI — Wan2.2 video tutorial](#) (accessed 2026-07-07); [ComfyUI — server API routes](#) (verified 2026-07-07 via the image-gen sibling, same engine).
4. **LTX-Video is the genuine fast/co-reside fallback: DiT, distilled, low-VRAM.** Upstream: “LTX-Video is the first DiT-based video generation model that contains all core capabilities of modern video generation in one model”; the ltxv-2b-0.9.8-distilled variant is “15× faster” and “real-time

capable”, with “New default resolution and FPS: 1216×704 pixels at 30 FPS”, generating “720x480x121 videos in under a minute on RTX 4060 (8GB VRAM)” and a LoRA distilled variant that “Requires only 1GB of VRAM”. Its footprint at fp8+tiling (~6–8 GB (EST)) fits the free 12.7 GB with the broker’s 2 GiB headroom (§1.5). **Honest nuance (§11.4.6):** LTX’s own desktop system-requirements page lists a “*minimum 32GB+ VRAM*” floor — that is for the FULL 1txv-13b-0.9.8-dev / default-config lane, NOT the distilled 2B quantised lane the RTX-4060 8 GB report describes; the 13B-dev lane is therefore a **full burst**, the 2B-distilled lane is the **co-reside fast lane**. Sources: [Lightricks/LTX-Video \(GitHub README\)](#) (accessed 2026-07-07); [LTX-Video — system requirements](#) (accessed 2026-07-07).

5. **The three-component VRAM model is why “co-reside video” is offload-dependent.** Video diffusion has three independently-VRAM-consuming components — the text encoder (WAN’s umt5-xxl $\approx$ 9.4 GB fp16 / T5-XXL-class, community-reported), the denoising backbone, and the VAE decoder. The documented sub-12.7 GB paths all combine: **text-encoder** → **CPU** (removes ~9 GB from VRAM), **backbone fp8/GGUF-q4** (halves the backbone), and **VAE temporal/spatial tiling** (one tile in VRAM at a time). This is the exact stack that lets WAN-5B/LTX-2B/Hunyuan-fp8 fit small cards, and the evidence base for “flagship quality $\Rightarrow$ full burst; distilled/5B = co-reside-tight.” Sources: community low-VRAM guides surfaced 2026-07-07 ([Will-It-Run-AI video-gen GPU guide](#), [Will-It-Run-AI Wan 2.2 VRAM guide](#)) — treated as (EST – measure), not upstream.
6. **Why HunyuanVideo is rejected as the default (compared, kept selectable).** It is “a 13B 3D DiT model requiring 47–58 GB VRAM at FP16” — the heaviest of the three; ComfyUI can run it “on GPUs with only 8GB VRAM through temporal tiling” at reduced resolution, but slowly, and it is governed by the “Tencent Hunyuan Community License” which “permits research and personal use but restricts commercial deployment”. Heaviest + non-commercial $\Rightarrow$ never the default, mirroring the image sibling’s FLUX-dev licence caveat (§1.4 Q). Sources: [Will-It-Run-AI HunyuanVideo VRAM guide](#) + [ComfyUI blog — running Hunyuan with 8GB VRAM](#) (both accessed 2026-07-07).
7. **Why the hosted-API lane (E) is rejected as default.** A hosted video API requires a committed key (§CONST-042 / §11.4.10 secret-leak surface) + third-party egress of user prompts, and defeats the offline/self-host posture — AND OpenAI’s own Sora Videos API is “*deprecated and will shut down on September 24, 2026*” (§3 note), so a hosted default would be unstable. It MAY be added later as an opt-in, config-injected, key-from- .env provider descriptor — never the default (mirrors IMAGE_GEN_PROVIDER.md §1.3(7)).

1.4 Model selection (config value, never hardcoded — §CONST-046 / §11.4.35)

Model (ComfyUI)	Params	Licence	Fits free ~12.7 GB?	Wall-clock / 5 s clip (EST — measure)	Role
LTX-Video 2B-distilled (fp8 + tiling, umt5/T5 on CPU)	2B	OpenRAIL-M	Yes (~6–8 GB (EST)) → co-reside	seconds–tens of seconds (<i>“real-time capable”, “under a minute on RTX 4060”</i>)	Default fast/co-reside fallback — video with no coder pause
WAN 2.2 TI2V-5B (fp16 + ComfyUI native offload)	5B	Apache-2.0	Yes-TIGHT (~8 GB (EST), offload-dependent)	minutes (<i>“under 9 min single consumer GPU”</i> ; 5090 faster (EST))	Default lighter primary — commercial-clean, co-reside-tight
WAN 2.2 T2V-A14B / I2V-A14B (fp8_scaled MoE, both experts)	27B/14B-active	Apache-2.0	No (~14–20 GB (EST) peak) → full burst	many minutes (I2V-A14B ~1 hr 20 min RTX 4090 (EST); 5090 faster)	Primary quality lane — full burst (coder paused, operator-gated Q6)
LTX-Video 13B-dev (fp8)	13B	OpenRAIL-M	No (docs floor “32GB+”) → full burst	~tens of seconds (<i>“HD in 10 s on H100”</i>)	Higher-quality LTX lane — full burst
HunyuanVideo (13B 3D-DiT,	13B	Non-commercial	fp8+tiling ~8 GB		Compared, rejected-as-

Model (ComfyUI)	Params	Licence	Fits free ~12.7 GB?	Wall- clock / 5 s clip (EST – measure)	Role
fp8 + temporal tiling)		(Tencent Hunyuan Community License)	reduced- res (slow); fp16 47–58 GB → burst	slow (minutes)	default; research/ quality burst only

Licence note (§11.4.6, flag for review — Q5): WAN 2.2 (all lanes) is **Apache-2.0** — commercial-clean, so unlike the image sibling’s FLUX-dev caveat, WAN’s quality lane is NOT licence-blocked. LTX-Video is **OpenRAIL-M** (use-based restrictions — review the RAIL clauses for the product’s use cases). **HunyuanVideo is non-commercial** — blocked for commercial ship, kept as a research-only selectable model. Documentation flag, not a resolution.

Recommendation: default fast/co-reside LTX-Video 2B-distilled (fastest, fits) AND WAN 2.2 TI2V-5B (Apache-2.0, 720P) as the everyday lanes; WAN 2.2 A14B (or LTX-13B-dev) as the quality lane behind the same route on a scheduled full burst — chosen on measured Budget () . free at admission time (§2), never a static guess.

1.5 VRAM budget — ESTIMATES to be measured (§11.4.6), fit-vs-burst verdict

All figures (EST – measure); replace with on-card nvidia-smi deltas + p50/p95/p99 wall-clock captured under docs/qa/<run-id>/video_gen/ before any PASS. Free-VRAM baseline: coder resident ⇒ ~12.7 GB free (§0). Broker headroom = 2 GiB (broker.go:13 HeadroomBytes), so a co-reside model must fit need + 2 GiB ≤ 12.7 GiB ⇒ need ≤ 10.7 GiB.

Config	Denoiser backbone (EST)	Text enc (umt5/T5-XXL) (EST)	VAE + working (EST)	Peak resident (EST)	Verdict vs free 12.7 GB
WAN 2.2 A14B fp16 (reference)	very large	~9.4 GB (umt5 fp16)	large	≥ 80 GB (upstream: “at least 80GB VRAM”)	Full burst + quant — far exceeds 32 GB card at fp16
			~2 GB	~14–20 GB	

Config	Denoiser backbone (EST)	Text enc (umt5/T5-XXL) (EST)	VAE + working (EST)	Peak resident (EST)	Verdict vs free 12.7 GB
WAN 2.2 A14B fp8_scaled (both experts, umt5 resident)	~10–14 GB	~4.7 GB (umt5 fp8)			Full burst (coder paused) — over 12.7 GB free, fits 5090's 32 GB
WAN 2.2 TI2V-5B fp16 (ComfyUI native offload, umt5 offloaded)	~5–6 GB	~0.5 GB GPU-resident (umt5 offloaded)	~1.5 GB	~7–9 GB	Co-reside ✓-TIGHT (ComfyUI: “ <i>fit well on 8GB ... native offloading</i> ”)
LTX-Video 2B-distilled fp8 + tiling (text enc on CPU)	~3–4 GB	~0.3 GB (CPU-offloaded)	~1.5 GB (tiled)	~6–8 GB	Co-reside ✓ (“720x480x121 ... on RTX 4060 (8GB)”)
LTX-Video 13B-dev fp8	large	~4.7 GB	~2 GB	> 12.7 GB (docs floor “32GB+”)	Full burst
HunyuanVideo fp8 + temporal tiling (reduced res)	~5–6 GB	~4.7 GB	~1.5 GB (tiled)	~ 8–10 GB (slow)	Co-reside-TIGHT but SLOW + non-commercial → not default

Latency (EST) — the load-bearing difference from image-gen: LTX-2B-distilled seconds–tens-of-seconds; WAN TI2V-5B minutes (upstream “*under 9 min*” on a slower card); WAN A14B / LTX-13B / Hunyuan many minutes to ~1 hr on consumer cards. **These numbers gate nothing until measured**, but they establish that a video job holds the single-owner burst FAR longer than an image job — so the route MUST be async (§3) and the burst queue’s starvation guard (VRAM_BROKER.md §5) is load-bearing.

2. VRAM-broker integration — the burst lease (§11.4.119)

The broker CORE **already exists** — `internal/vrambroker/broker.go` + `errors.go` (commit `a12df57`, verified 2026-07-07). Video-gen is class **ClassVideo** (`broker.go:22` `ClassVideo Class = "video" // burst tier – single-owner (§11.4.119)`), and `Class.IsBurst()` is true for it (`broker.go:29` `return c == ClassImage || c == ClassVideo`). The provider acquires a burst lease around every generation job:

```
// pseudo-flow in the video-gen provider / async job worker
lease, err := broker.Acquire(ctx, vrambroker.ClassVideo,
    needBytes) // needBytes = measured model footprint
if err != nil {
    // ErrBurstInUse      → an image OR video burst is already
    // live → 429/queue (§2.2)
    // ErrBudgetExceeded → over budget while coder resident →
    // the operator-gated
    // "pause warm tier" decision (§2.3),
    // NOT an OOM (fail-closed)
    // ErrThermalUnsafe / ErrBudgetUnavailable → 503, honest
    // (§11.4.133)
    return mapBrokerErrorToJobFailure(err)
}
defer lease.Release() // idempotent (broker.go:47) – frees the
    single-owner slot
mp4 := backend.GenerateVideo(ctx, req) // ComfyUI /prompt (WAN/
    LTX graph) ... poll /ws ... /view
```

2.1 What the existing broker already enforces (reuse, don't reimplement §11.4.74)

- **Single-owner burst (§11.4.119) — image AND video share the ONE burst slot.** Acquire refuses a second burst lease with `ErrBurstInUse` **before** the budget read (`broker.go:147-149`) — and because `IsBurst()` is true for BOTH `ClassImage` and `ClassVideo` (`broker.go:29`), a video job and an image job **cannot** run concurrently, exactly the §11.4.119 partitioning the plan requires. A queued video job behind a live image burst (or vice-versa) is rejected deterministically.
- **Fail-closed over-budget refusal (NEVER an OOM).** Admission is `admit(free, needBytes, headroom) = free >= needBytes + headroom` on the **measured** `nvidia-smi free` (`broker.go:161-183`, `budget.go:readNvidiaSMI`); an unreadable budget returns `ErrBudgetUnavailable` (fails closed, `broker.go:157-160`); an over-budget

request returns `ErrBudgetExceeded` (`broker.go:161-163`, `errors.go:12` “*request exceeds available VRAM budget (refused, fail-closed to avoid OOM)*”). This is the anti-bluff core the §1.1 mutation disables.

- **Thermal/power safety gate (§11.4.133).** With `ThermalGuard` injects a temperature/power probe consulted before admission (`broker.go:99,151-154`; `errors.go:28 ErrThermalUnsafe`). Video-gen is the **most sustained-power** workload in the fleet (minutes of full-card compute) — this gate is the most load-bearing here of any capability.
- **Idempotent release.** `Lease.Release()` frees the single-owner slot exactly once (`broker.go:47-55`) — a `defer lease.Release()` around the (long) job hands the card back deterministically even on a mid-generation error/cancel.

2.2 Co-reside (fast lane) — no coder pause needed

For the **co-reside fast lane** (LTX-2B-distilled / WAN TI2V-5B, `needBytes` ≈ 8 GiB), `Acquire(ctx, ClassVideo, 8 GiB)` **succeeds while the coder stays resident**: $8 + 2 \leq 12.7 \Rightarrow$ `admit` returns true. It is still a **single-owner** burst (no concurrent image/video), but does **not** require pausing the coder — the job runs in the free residual and `Release` hands it back. This is the lane that ships first (no operator gate). Because it is TIGHT (§1.5), the provider **MUST** size `needBytes` from a **measured** footprint and re-read `Budget()`. `free` at admission — a co-reside fit that held for a small clip may not hold for a longer/higher-res one.

2.3 Pause-warm-tier (quality lane) — operator-gated, DESIGN-only

For the **quality lane** (WAN A14B / LTX-13B fp8, `needBytes` $\approx 16-20$ GiB), `Acquire(ctx, ClassVideo, 18 GiB)` **fails closed with `ErrBudgetExceeded`** while the coder is resident ($18 + 2 > 12.7$). The design intent (`VRAM_BROKER.md` §2/§4: “*may require pausing warm tier ... pause the warm tier, run, and the warm tier resumes*”) is to evict/park the warm+coder tier to free the card, run the burst, then resume.

Honest boundary (§11.4.6): the broker does NOT yet implement eviction. The shipped `Acquire` **refuses** over-budget rather than pausing the coder (reading `broker.go`, there is no `sleep(coder) / vLLM-Sleep-Mode / keep_alive:0` call). Pausing the resident coder fleet to free $\sim 18-30$ GB is a **high-blast-radius, operator-gated decision** (it stalls every live agent, and — because a video burst lasts MINUTES — the stall is long), so it is **deferred to the P1-T4 residency-scheduler implementation + an explicit operator go**, per §0. **This is the SAME open question as the image sibling’s Q6** (`IMAGE_GEN_PROVIDER.md` §2.3 / Q6 pause-warm-tier deferral) — video inherits it, and makes it sharper because the occupancy is longer. Until P1-T4 lands, the quality-lane path is admissible only during a scheduled burst window where

the coder tier is already parked; the co-reside fast lane (§2.2) covers the everyday case. Per §11.4.101 block-only-when: pausing the coder for minutes is irreversible-for-the-session + high-blast-radius, so it blocks on the operator rather than being decided autonomously.

2.4 Async job model (composes the broker queue) — mandatory here

A video burst **always** takes minutes (§1.5) and may **wait** (for a scheduled window, or behind a live image/video burst), so the gateway route (§3) is **async-only** (submit → poll → download) — there is NO synchronous path (unlike image-gen, which offers a fast sync path for its 2–8 s co-reside lane). A queued job surfaces honestly (202 Accepted + a job id; 429 with Retry-After on ErrBurstInUse; the job's own status field transitions queued → running → succeeded | failed), never a silent multi-minute HTTP hang (VRAM_BROKER.md §5 starvation guard). The async-job discipline is the plan's explicit P3-T3 shape ("*async job endpoint*").

3. API contract — async video-job route (consistent with API_CONTRACT.md + the OpenAI Videos API async shape)

HelixLLM registers **no** video route today — `grep -rn "videos/generations\|/v1/videos\|ClassVideo" submodules/helix_llm/internal --include=*.go` (excluding the broker enum) returns **empty** (verified 2026-07-07). This provider **adds** an async video-job route under the gateway /v1 group so it inherits the API-key middleware + rate-limit + security-headers uniformly (API_CONTRACT.md §2/§3) — the same reviewed placement the image + translation spikes chose. The route shape mirrors the **OpenAI Videos API async workflow** ("*create → poll status → download when ready*") — the current latest-source async video contract (§11.4.99).

3.1 Create the job — returns a job id, NEVER blocks

POST /v1/videos

Authorization: Bearer <key>

gateway /v1 API-key middleware

Content-Type: application/json

{

"model": "helix-video",

Helix alias → backing WAN/LTX mod

"prompt": "a red fox trotting through autumn leaves, cinematic, slow pan

"seconds": 5,

OPTIONAL — clip duration

"size": "1280x704",

OPTIONAL — e.g. 1280x704, 1216x704

```

    "fps": 24, # OPTIONAL – target frame rate
    "input_reference": null # OPTIONAL – an image (base64/url) f
  }

→ 202 Accepted
{
  "id": "video_01H...", # job id – poll this
  "object": "video",
  "model": "helix-video",
  "status": "queued", # queued | running | succeeded | fai
  "created_at": 1751840000
}

```

- **model** — a **Helix alias**, not a raw HF id (CONST-036/037: models come from the provider layer / LLMsVerifier, never hardcoded). Alias → backing-model map (helix-video → LTX-2B-distilled / WAN-TI2V-5B / WAN-A14B) lives in HelixLLM config (env/YAML), never a source literal (§CONST-046) — adding a model = a config edit, no code change.
- On `broker.Acquire` returning `ErrBurstInUse`, `create` returns **429** + **Retry-After** (another image/video burst is live) — the client re-submits; the job is never silently dropped.

3.2 Poll the job — status transitions

```

GET /v1/videos/{id}
Authorization: Bearer <key>

```

```

→ 200 OK
{ "id": "video_01H...", "object": "video", "status": "running", "progress"

```

`status` ∈ {`queued`, `running`, `succeeded`, `failed`}; `progress` is derived from the ComfyUI `/ws` progress events. A failed job carries an OpenAI-error-shaped error object (§3.4), never a partial/stub artefact. Poll interval ≥ 5 s (recommended 10–20 s), consistent with the OpenAI Videos guidance and the minutes-scale wall-clock (§1.5).

3.3 Download the produced mp4

```

GET /v1/videos/{id}/content
Authorization: Bearer <key>

```

```

→ 200 OK (Content-Type: video/mp4) # the produced clip, streamed

```

The success artefact is a real `.mp4` (H.264, faststart) — never a returned stub clip (the bluff §5 kills). `response_format` alignment: `b64_json` inline (for small clips /

self-contained) OR the /content byte-stream (default for video, since clips are larger than images); a url lane requires a configured static-asset host (Open Q3, injected per §CONST-045).

- GET /v1/models MUST advertise helix-video with owned_by: "helix" and a video capability flag so LLMsVerifier + clients discover it (CONST-036/040) — the capability flag is verifier-sourced, never hardcoded (P2-T3 seam).

3.4 Backend bridge — ComfyUI (WAN 2.2 / LTX-Video graph)

- The async job worker builds a WAN/LTX **workflow graph** from {prompt, seconds, size, fps, input_reference} → POST /prompt (→ prompt_id) → waits on the /ws executed event (or polls GET /history/{prompt_id}, translating ComfyUI progress → the job progress) → fetches the mp4 via GET /view → serves it via /v1/videos/{id}/content. (ComfyUI routes are the same the image sibling verified — §1.3(3).)
- Error shape: OpenAI-error envelope, consistent with the gateway (API_CONTRACT.md §3): {"error": {"message": "...", "type": "invalid_request_error"}} for auth/validation; **429 + Retry-After** on ErrBurstInUse; a failed job with {"...", "type": "server_error"} on ErrBudgetExceeded / ErrThermalUnsafe / backend-not-warm — **never** a returned stub/black clip.

§11.4.99 negative-finding note. The /v1/videos async triple (create → poll → /content) is grounded in [OpenAI — Sora video-generation guide](#) + [OpenAI — Create video API reference](#) (accessed 2026-07-07). OpenAI's HOSTED Sora Videos API + sora-2 models are themselves *“deprecated and will shut down on September 24, 2026”* — but **we mirror only the async-job SHAPE**, serving local WAN/LTX via ComfyUI, so the deprecation of OpenAI's hosted models does not affect this contract. The canonical platform.openai.com/docs/api-reference returned HTTP 403 to automated fetch (same as the image sibling); the field list is grounded in the developers.openai.com mirror + the in-tree API_CONTRACT.md gateway placement.

4. Containerization — rootless podman via the containers submodule, WITH GPU

Per §11.4.76 (containers-submodule mandate) + §11.4.161 (rootless runtime) the service boots **through** vasic-digital/containers (pkg/boot / pkg/compose / pkg/health), never a hand-run podman/docker command, and never rootful. It is

the **same ComfyUI container the image sibling stands up** (§11.4.74) — video adds only the WAN/LTX model weights + workflow graphs, not a new engine.

4.1 Image + run shape (illustrative — config-injected, no hardcoded host §CONST-045)

- **Image:** the P0-T2 pinned CUDA-12.8 / sm_120 base (nvidia/cuda:12.8.x, 04_implementation_plan.md:46) + ComfyUI + the WAN 2.2 / LTX-Video / (optional HunyuanVideo) custom nodes. Pinned by digest in production (§11.4.76 clause 2). This is the SAME image profile as image-gen — one ComfyUI container serves both ClassImage and ClassVideo bursts (never concurrently, §2.1).
- **GPU passthrough:** the run spec **DOES** include `--device nvidia.com/gpu=all --security-opt=label=disable` — the exact P0 GPU-passthrough proof (04_implementation_plan.md:44). Structural reason it depends on **P0** (host GPU foundation) and **P1** (the broker).
- **Model source:** WAN/LTX/umt5/VAE weights are a §11.4.77 re-obtain artefact (gitignored; a `fetch_weights.sh`-class script downloads from HF — matches 04_implementation_plan.md P7-T1). Video weights are LARGE (WAN A14B two experts + umt5-xxl + VAE » image weights) — the `.gitignore-meta/<slug>.yaml` MUST record the disk budget honestly (§11.4.77). `$MODELS_DIR` is **injected** (env), mounted read-only (`-v $MODELS_DIR:/models:ro`).
- **RAM budget (video-specific):** the CPU-offloaded umt5-xxl text encoder needs **host** RAM (upstream/community: “*at least 24GB system RAM*” for WAN; LTX system-requirements “*32GB system memory*” min, “*64GB+*” recommended) — the container’s host-memory reservation MUST account for the offloaded encoder, and §12.6 host-memory safety applies so a video burst does not starve the developer host.
- **Port:** a config-injected host port (ComfyUI :8188, shared with image-gen since one container serves both), reached by the HelixLLM gateway.
- **Boot is part of the test entry point** (§11.4.76 on-demand-infra invariant): the HelixQA video bank boots the container via the submodule, waits on `pkg/health` (ComfyUI / `system_stats`), acquires the ClassVideo burst lease, then drives the `async` route — a short-circuit fake that skips the boot is a §11.4 violation.
- **Catalogue-Check (§11.4.74):** extend `vasic-digital/containers@<sha>` — the image-gen ComfyUI compose profile is extended (not duplicated) with the WAN/LTX model set + node deps; never an in-project ad-hoc compose file.

4.2 Cross-platform + resource/host-safety hygiene

- GPU passthrough is Linux/NVIDIA-specific; on a host without the 5090 the bank SKIPS-with-reason (`hardware_not_present`, §11.4.69) — never a faked

- PASS. Video has **no** CPU-only degraded lane worth shipping (CPU video diffusion is impractically slow) — an honest SKIP, not a slow-CPU fallback.
- Container VRAM/compute bounded by the broker lease (§2) + pkg/health; §12.6 host-memory + §11.4.133 thermal safety apply — a MINUTES-long full-power video burst must not starve the host nor thermally endanger the card (the broker’s ThermalGuard is the most load-bearing here of any capability, §2.1).
-

5. Anti-bluff acceptance — the ONE machine-checkable runtime signature (§11.4.108)

Definition of done for this provider: on a **clean deploy** (§11.4.108/§11.4.139 — container freshly booted via the containers submodule, ClassVideo burst lease granted, gateway pointed at it), the following single machine-checkable signature verifies and is captured to docs/qa/<run-id>/video_gen/:

RUNTIME SIGNATURE (video-gen: real, LIVE, prompt-faithful clip). Submit a **prompt pair** to POST /v1/videos — a **matched** prompt P = “a red fox trotting through autumn leaves, cinematic, slow pan” and an **unrelated control** prompt U = “a bowl of ramen steaming on a wooden table” — poll each to succeeded, download each mp4 via /v1/videos/{id}/content, and assert **ALL** of: 1. **Real mp4, correct container/codecs/geometry (ffprobe):** the mp4 demuxes; nb_frames is **non-zero and equals round(seconds × fps)** within tolerance; codec_name, width×height, and avg_frame_rate match the request (the plan’s “ffprobe codec/resolution/fps” made mechanical + count-exact). 2. **LIVE, ADVANCING frames — NOT a single/frozen/black clip (§11.4.107(1)(2)):** a **freeze-detection oracle** over the whole clip — perceptual-hash adjacent-frame distance (ffmpeg freezedetect-class) above a fixture floor for a steady-state window — AND an **independent frame-advance check** (decoded PTS strictly increasing, no all-duplicate run). A static, black, or single-frame-repeated clip FAILS (byte-identity is only a zero-cost pre-filter, per §11.4.107). This is the native §11.4.107 case: video liveness is the point. 3. **Prompt → video correspondence (semantic):** sample **K** frames uniformly and compute CLIPScore(frame_k, P) (or a VLM caption of the sampled frames vs P, using the P3-T1 Qwen-VL sibling); assert the aggregate ≥ a **floor** calibrated on the project’s own fixtures (§11.4.107(13)) — the clip genuinely depicts its prompt. 4. **Semantic-order margin (anti-wrong-content / anti-noise guard):** CLIPScore(frames_P, P) − CLIPScore(frames_P, U) ≥ margin — the clip matches its OWN prompt more than an unrelated caption (a wrong-prompt, cached-from-a-different-request, or noise clip fails this even if it passes 1–2). 5.

Decoder-health census (§11.4.136): no dropped/corrupt frames beyond a numeric budget; no PTS re-order/discard; the demuxer reports zero decode errors. The captured artefact is the raw create/poll/content transcript + the decoded mp4s + the ffprobe report + the per-frame CLIPScore matrix + the freeze/advance report + PASS/FAIL verdict with its evidence path (feature class `video_display/gpu_render`, §11.4.69).

CLIPScore is “a metric based on a modified cosine similarity between representations for the input image and the caption” — **reference-free**, needing no golden video — applied per sampled frame; the freeze-detection + frame-advance oracle is the video-native §11.4.107 liveness battery. Together they can only PASS if a real diffusion model produced a **live, prompt-faithful** clip through the real async gateway route + a live ClassVideo burst lease — impossible to satisfy with a returned stub, a static/black clip, a frozen single frame, or a wrong-prompt/noise clip. CLIPScore source: [CLIPScore — reference-free image-text similarity \(Hessel et al.\)](#) (accessed 2026-07-07).

5.1 Golden-good / golden-bad self-validation (§11.4.107(10))

The analyzer (ffprobe + freeze/advance + CLIPScore) **is itself mutation-proofed** with a fixture set, wired into the meta-test:

- **golden-good fixture** — a captured real, live, prompt-faithful clip + its prompt where every criterion genuinely holds → the analyzer **MUST** return **PASS**.
- **golden-bad fixtures** (each **MUST** return **FAIL**, proving the analyzer cannot be fooled):
 1. **static / all-black clip** (or a single frame duplicated N times) → fails the liveness / frame-advance check (criterion 2) — the canonical §11.4.107 stale/ frozen-frame bluff.
 2. **frozen-tail clip** — real motion for the first frames then a frozen freeze to length → fails freeze-detection over the steady-state window (criterion 2).
 3. **random-noise clip** → passes ffprobe geometry but fails the prompt-correspondence floor + the semantic-order margin (criteria 3+4).
 4. **wrong-prompt clip** (a real fox clip returned for the ramen prompt, or vice-versa) → fails the semantic-order margin (criterion 4) — catches a mis-wired backend / a cached-from-a-different-request clip.
 5. **wrong-geometry / short-count clip** (wrong fps, resolution, or `nb_frames ≠ round(seconds×fps)`) → fails criterion 1.

Paired §1.1 mutation: strip the **freeze/frame-advance** (or the semantic-order-margin) assertion from the analyzer → the static-clip / wrong-prompt golden-bad

fixture PASSes → the gate FAILs. That mutation is the mechanical proof the acceptance test is not itself a bluff (§11.4.107(10)).

5.2 Higher-order + resilience proofs (compose, do not replace §5)

- **Metamorphic seed determinism (§11.4.50):** same prompt + same seed → CLIPScore-identical (within tolerance) clip across two calls; different seeds → different clips with both matching the prompt; a seconds/fps change scales nb_frames proportionally (a metamorphic relation on the geometry check).
- **Re-runability (§11.4.98):** the whole bank PASSes at -count=3 with self-cleaning state (burst lease released between runs).
- **Stress + chaos (§11.4.85):** $N \geq 10$ sequential jobs (the burst queue serialises them — throughput + p50/p95/p99 wall-clock captured, which for video is MINUTES-scale); a **second concurrent** image/video request **MUST** hit ErrBurstInUse → 429 (the §11.4.119 single-owner proof, shared with image-gen); chaos = container SIGKILL mid-generation → broker lease auto-releases (or the P1-T4 lease-TTL reaps it), the async job transitions to failed with an honest error, the card is not leaked; boundary prompts (empty, max-length, unicode, adversarial, seconds=0/max) each categorised.
- **Broker-refusal negative case (composes VRAM_BROKER.md §6):** a scripted over-budget quality-lane Acquire returns ErrBudgetExceeded and the job fails honestly — it does **NOT** OOM the card (fail-closed); paired §1.1 mutation (disable admit) → the over-commit would be granted → the broker gate FAILs.
- **Feature-class evidence (§11.4.69):** the closed sink-side taxonomy has a video_display class; this provider maps to it (also gpu_render for the compute side). Evidence shape = the captured ffprobe + freeze/advance + CLIPScore artefact above — never a single screenshot / single frame (explicitly the §11.4.107 bluff this signature forbids).

5.3 Four-layer verification (§11.4.108)

1. **SOURCE** — the /v1/videos async triple (create/poll/content) + the WAN/LTX ComfyUI graph builder + ClassVideo Acquire/Release wiring + provider descriptor committed; pre-build grep gate.
2. **ARTIFACT** — the ComfyUI image pulled + pinned by digest; the WAN/LTX/umt5/VAE weights present (fetch_weights.sh); pkg/health green; the sm_120 GPU build present (P0-T3).
3. **RUNTIME-ON-CLEAN-TARGET** — the §5 signature verifies against a freshly-booted container **with a live ClassVideo burst lease** (not a stale one) — the definition of done. **Ready-to-run the moment a burst window is scheduled** (the operator go of §0/§2.3), or immediately for the co-reside fast lane (§2.2).

4. **USER-VISIBLE** — an OpenAI-Videos-compatible client (HelixCode / any CLI agent) POSTs /v1/videos, polls, downloads a real prompt-faithful clip that genuinely plays, and a downstream consumer renders it.
-

6. Open questions (resolve before coding)

- **Q1 Build order** — ship the **co-reside fast lane FIRST** (LTX-2B-distilled and/or WAN TI2V-5B — available today, no operator gate, §2.2) and defer the full-burst WAN-A14B / LTX-13B quality lane to a scheduled window? (Leaning: yes — a real, usable video lane now beats a blocked flagship, exactly as the image sibling chose.)
- **Q2 Which co-reside default** — **LTX-2B-distilled** (fastest, OpenRAIL-M) or **WAN TI2V-5B** (Apache-2.0, 720P, tighter fit)? Decide on measured Budget () . free fit + wall-clock + licence fit for the product.
- **Q3 /content byte-stream vs b64_json inline vs a url lane** — a url lane requires a configured static-asset host (§CONST-045 injected, never hardcoded); /content (default) needs none. Confirm the client's preferred download shape.
- **Q4 Does the containers ComfyUI compose profile** (stood up for image-gen) already carry the WAN/LTX custom nodes + weight-fetch, or is this a \$11.4.74 extend PR adding the video model set? (Investigate before scaffolding.)
- **Q5 Licence** — WAN 2.2 is Apache-2.0 (clean); LTX-Video is OpenRAIL-M (use-based restrictions — review the RAIL clauses); HunyuanVideo is non-commercial (kept research-only). Reconcile the LTX OpenRAIL-M use-restrictions against HelixLLM's distribution model before making LTX the commercial default.
- **Q6 Broker eviction (§2.3)** — **SAME open question as the image sibling's Q6.** The shipped broker **refuses** over-budget rather than pausing the coder. When P1-T4 implements the residency scheduler (vLLM Sleep Mode / keep_alive:0), the quality-lane “pause warm tier → run → resume” path becomes autonomous within a scheduled window; until then it is operator-gated. Video sharpens it: the pause is MINUTES-long (§1.5), so a longer coder stall than image-gen's.
- **Q7 Content oracle** — per-frame **CLIPScore** (consistent with the image sibling) vs a **VLM caption** of sampled frames (using the P3-T1 Qwen-VL sibling, which understands motion/temporal context better). Decide on measured golden-good/bad separation on the project's own fixtures (§11.4.107(13)). K (frames sampled) is a calibrated parameter, not the estimate here.
- **Q8 Async job persistence** — where do in-flight/queued job records live (in-memory vs a small store) so a gateway restart mid-burst doesn't orphan a job? (Composes the §11.4.147 crashed-agent-respawn discipline at the job layer.)

7. Composition footer — constitutional anchors touched

- **§11.4.6** (no-guessing) — every VRAM/latency/wall-clock figure flagged (EST – measure); the licences, the broker-does-not-yet-evict boundary, the LTX 8 GB-vs-32 GB nuance, and the co-reside-vs-burst verdicts all flagged, not asserted.
- **§11.4.74** (extend-don't-reimplement) — reuse ComfyUI (the SAME engine the image sibling stands up) + WAN 2.2 + LTX-Video (the models the plan names) + the EXISTING internal/vrambroker core (ClassVideo) + extend the containers ComfyUI profile; no bespoke video server, no re-implemented admission gate.
- **§11.4.76 / §11.4.161** (containers submodule / rootless) — boot via pkg/ boot+pkg/compose+pkg/health, rootless podman, WITH the GPU device.
- **§11.4.77** (re-obtain mechanism) — WAN/LTX/umt5/VAE weights gitignored + `fetch_weights.sh`, with an honest LARGE disk budget in `.gitignore-meta`.
- **§11.4.81** (cross-platform parity) — GPU passthrough Linux/NVIDIA-specific; no-GPU host $\Rightarrow$ honest `hardware_not_present SKIP`; no CPU-only video lane (honest SKIP, not a slow-CPU pass-off).
- **§11.4.99 / §11.4.150** (latest-source + deep multi-angle research) — engine decision cited to LATEST upstream docs across ≥ 5 distinct angles (WAN 2.2 repo + ComfyUI WAN tutorial, LTX-Video repo + LTX system-requirements, HunyuanVideo VRAM/licence, OpenAI Videos async API, CLIPScore); OpenAI-Sora-deprecation negative finding documented.
- **§11.4.107** (AV liveness) — the §5 signature is a full liveness battery (freeze-detection + independent frame-advance), NOT a single frame / non-zero count; §11.4.107(10) golden-good/golden-bad self-validated analyzer; §11.4.107(13) fixture-calibrated floor/margin. Video is the native §11.4.107 case.
- **§11.4.108 / §11.4.139** (four-layer runtime-signature on a clean target) — the §5 acceptance signature is the definition of done, ready-to-run on a scheduled burst window; the clean-artifact precondition asserted.
- **§11.4.119** (single-resource-owner burst) — video-gen is a ClassVideo single-owner burst; concurrent image/video refused with `ErrBurstInUse (IsBurst())` true for both classes).
- **§11.4.133** (thermal/power safety gates admission) — the broker's ThermalGuard is MOST load-bearing here (minutes of sustained full-power compute).
- **§11.4.136** (real-content playback / decoder-health) — the §5 decoder-health census (drop-frame budget, no PTS re-order) applies to the produced mp4.
- **§11.4.85 / §11.4.98 / §11.4.50** (stress+chaos / full-automation / determinism).

- **§11.4.135** (standing regression guard) — the \$5 signature registers as a permanent guard.
- **§11.4.69** (sink-side evidence taxonomy) — maps to `video_display` + `gpu_render`.
- **CONST-036/037/040** (LLMsVerifier single source of truth; capability flags verifier-sourced) — `helix-video` alias + video capability from the verifier, never hardcoded.
- **CONST-042 / §11.4.10** (no-secret-leak) — the hosted-API lane rejected as default precisely because it requires a committed key + third-party egress.
- **CONST-046** (no hardcoded content) — model ids / host / port / alias map config-injected.

Sources verified

Deep-research 2026-07-07: - <https://github.com/Wan-Video/Wan2.2> - https://docs.comfy.org/tutorials/video/wan/wan2_2 - <https://github.com/Lightricks/LTX-Video> - <https://docs.ltx.video/open-source-model/getting-started/system-requirements> - <https://willitrunai.com/blog/hunyuanvideo-1-5-vram-requirements> - <https://blog.comfy.org/p/running-hunyuan-with-8gb-vram-and> - <https://willitrunai.com/blog/wan-2-2-vram-requirements> - <https://willitrunai.com/blog/video-generation-gpu-guide-2026> - <https://developers.openai.com/api/docs/guides/video-generation> - <https://developers.openai.com/api/reference/resources/videos/methods/create> - https://docs.comfy.org/development/comfyui-server/comms_routes (ComfyUI server routes — same engine as image-gen, verified via `IMAGE_GEN_PROVIDER.md` 2026-07-07) - <https://arxiv.org/abs/2104.08718> (CLIPScore — reference-free image-text similarity, applied per-frame)

(Negative finding, §11.4.99(B): OpenAI's HOSTED Sora Videos API + `sora-2` models are *deprecated, shutting down 2026-09-24* — we mirror only the `async-job` SHAPE, serving local WAN/LTX via ComfyUI, so the deprecation does not affect this contract. `platform.openai.com/docs/api-reference` returned HTTP 403 to automated fetch on 2026-07-07; the `/v1/videos` field list is grounded in the `developers.openai.com` mirror + the in-tree `API_CONTRACT.md` gateway placement. LTX-Video's desktop system-requirements floor of "*32GB+ VRAM*" is for the FULL 13B / default-config lane; the 2B-distilled quantised lane's "*RTX 4060 (8GB)*" report is the co-reside basis — both cited, the tension resolved explicitly in §1.3(4).)